Shiv Nadar University Chennai

CS3807 Deep Learning Laboratory

Experiment 2

Implementation of a Multi Layer Perceptron (MLP) for Multi Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 Deep Learning Laboratory	AY: 2026/27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

- An MLP have an input layer, some hidden layers and an output layer.
- The math is like this:

$$z^{(l)} = W^{(l)}a^{(l1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

- And for the output layer we use softmax:

$$\hat{y} = \text{Softmax}(z)$$

- The loss function is categorical cross entropy:

$$L = \sum_i y_i \log(\hat{y}_i)$$

- The architecture that was recommended to us is:

$$784 \rightarrow \text{Dense(128, ReLU)} \rightarrow \text{Dense(64, ReLU)} \rightarrow \text{Dense(10, Softmax)}$$

4. Dataset

- We used Fashion MNIST dataset.
- The details are:
  - Training Images: 60,000
  - Testing Images: 10,000

  - Classes: 10
  - Image Size: 28×28

5. Image Preprocessing

- Fashion MNIST images are stored as 28×28 matrices but dense layers need a one dimensional vector.
- So we flatten them:

$$28 \times 28 = 784$$

- For example a small 2x2 matrix like this:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

- The preprocessing steps we did:
  1. Load Fashion MNIST.
  2. Display some sample images.
  3. Flatten the images.
  4. Normalize pixels to [0,1].
  5. Convert labels to one hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output:

- Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

- We made an MLP with:
  - Input layer (784)
  - Dense(128, ReLU)
  - Dense(64, ReLU)
  - Dense(10, Softmax)
- And we displayed `model.summary()`.

Task 4: Model Training

- We compiled the model using:
  - Optimizer: Adam
  - Loss: Categorical Cross Entropy
  - Metric: Accuracy

- And trained for 20 epochs with batch size 32.

Task 5: Model Evaluation
- We computed:
  - Accuracy
  - Precision
  - Recall
  - F1 score
  - Confusion Matrix
  - Classification Report

7. Hyperparameter Optimization

- Instead of manually selecting hyperparameters we did automated hyperparameter optimization using RandomizedSearchCV with the SciKeras wrapper.
- GridSearchCV was also an option but we didnt use it.

The search space we defined is:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5 fold cross validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

What we actually did and why
- Ok so like the full search space was HUGE.
- If u actually multiply all the combinations it comes to like more than 11,000 which is just not possible to run in one lab session.
- So we decided to go with Randomized Search instead of Grid Search.
- Basically what that means is instead of checking every single combination we just randomly picked 20 combinations and tested those.
- Its not perfect coz we might miss the absolute best one but in practice it usually works pretty good coz only a few hyperparameters actually matter that much.
- So for each of the 20 random combinations we did 5 fold cross validation.

- That means we split the training data into 5 parts, train on 4 and test on 1, and rotate it 5 times.
- Then we take the average accuracy across all 5 folds.
- This gives a better idea of how well the model generalizes rather than just getting lucky on one test split.
- The best combination we found was actually pretty simple, just one hidden layer with 128 neurons and ReLU activation, no dropout.
- We trained it with Adam optimizer at learning rate 0.001, batch size 64, for 20 epochs.
- The average cross validation accuracy was 0.8903 which is 89.03%.
- Whats interesting is that the baseline model had TWO hidden layers (128 and then 64) but the optimized one only needed ONE hidden layer of 128 neurons.
- So basically a wider but shallower network worked better here, and its also faster to train so thats a win win.

Optimized Model Classification Report

Class	Precision	Recall	F1 score	Support
TShirt/top	0.90	0.77	0.83	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.83	0.79	0.81	1000
Dress	0.88	0.89	0.88	1000
Coat	0.79	0.85	0.82	1000
Sandal	0.97	0.96	0.97	1000
Shirt	0.66	0.76	0.71	1000
Sneaker	0.90	0.98	0.94	1000
Bag	0.98	0.96	0.97	1000
Ankle boot	0.98	0.91	0.94	1000
Accuracy		0.88		10000
Macro avg	0.89	0.88	0.88	10000
Weighted avg	0.89	0.88	0.88	10000

- Shirt is doing really bad honestly, precision is only 0.66 which means like one third of the stuff the model called "Shirt" wasnt actually a shirt.
- Pullover (0.83) and Coat (0.79) are also not great.
- But Trouser, Sandal, Bag and Ankle boot are all above 0.97 which is amazing.
- This makes sense coz if u look at the confusion matrix almost all the mistakes are between the upper body clothes, they look too similar when u flatten them into just a vector of numbers.

8. Mandatory Plots

1. **Sample Images.**

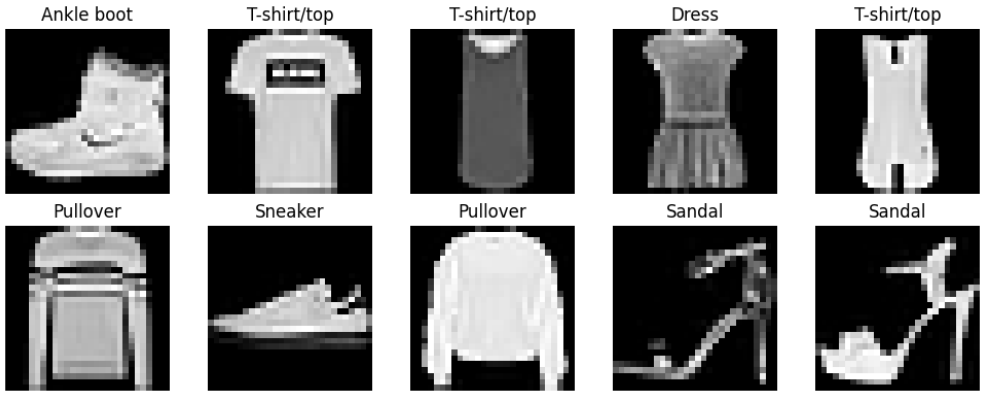

Figure 1: Grid of 10 sample images from Fashion MNIST dataset

- This was mostly just to check that the labels actually match the images coz its really easy to mess up the indexing and not realize.
- Also it was the first time we could see how similar some classes look, like coats and shirts look almost the same when u just see the grayscale, which explains why the model gets confused later.

2. **Class Distribution.**

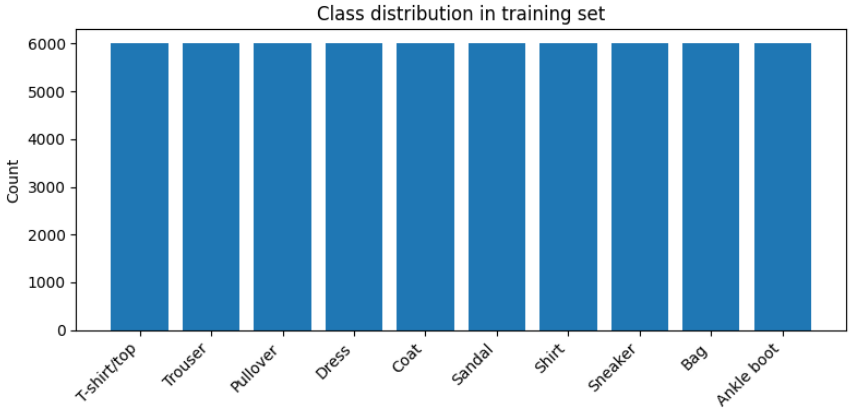

Figure 2: Bar chart showing class distribution (6000 images per class)

- The chart came out perfectly balanced, 10 classes with exactly 6000 images each.
- Since its so balanced we dont really have to worry about accuracy being misleading, unlike if one class had way more images than others.

3. **Training Accuracy vs Epoch.**

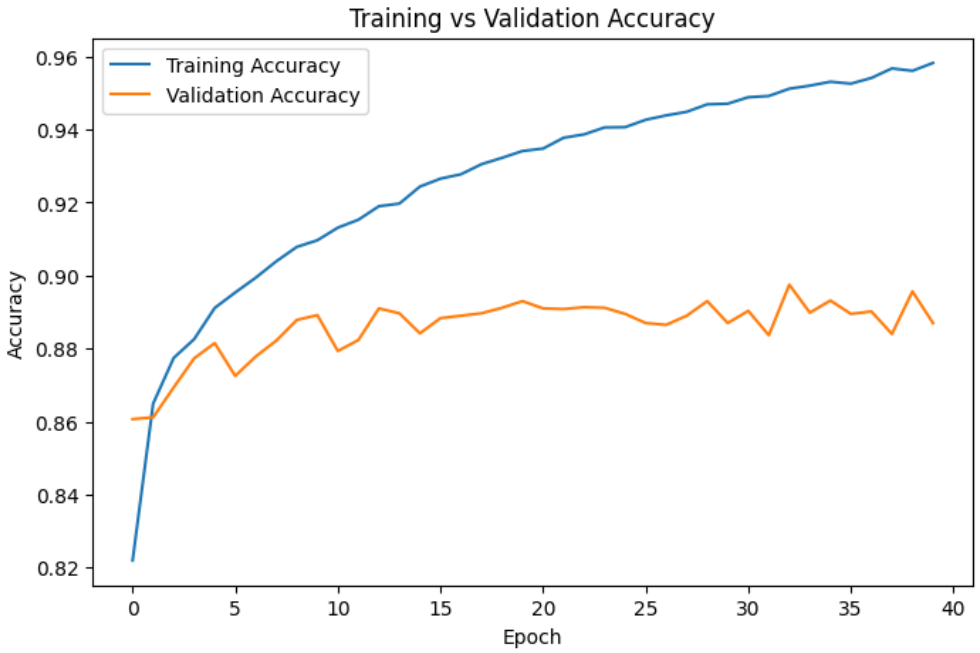

Figure 3: Training accuracy plotted against epochs

- Training accuracy kept going up for all 40 epochs and ended at 95.6%.
- It never really plateaued which honestly is kinda suspicious, usually means the model is just memorizing the training data instead of actually learning patterns.

4. **Validation Accuracy vs Epoch.**

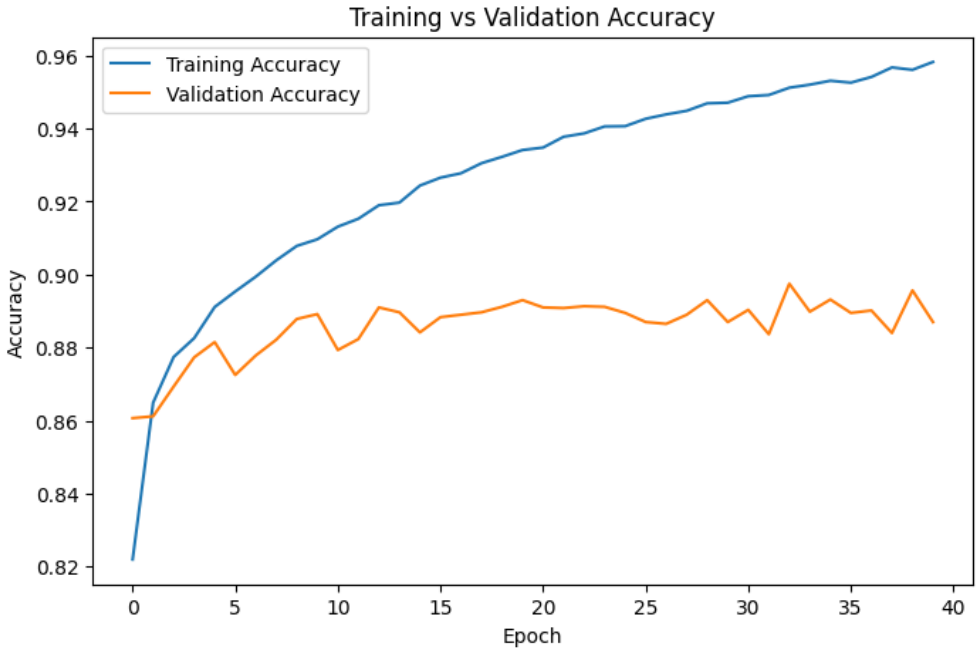

Figure 4: Validation accuracy plotted against epochs

- This one looks different from training.
- It went up fast in the first few epochs and then just stayed flat at around 88 to 89% from like epoch 7 onwards.

- So even tho training accuracy kept improving for 30 more epochs, validation barely changed, classic sign of overfitting.

5. **Training Loss vs Epoch.**

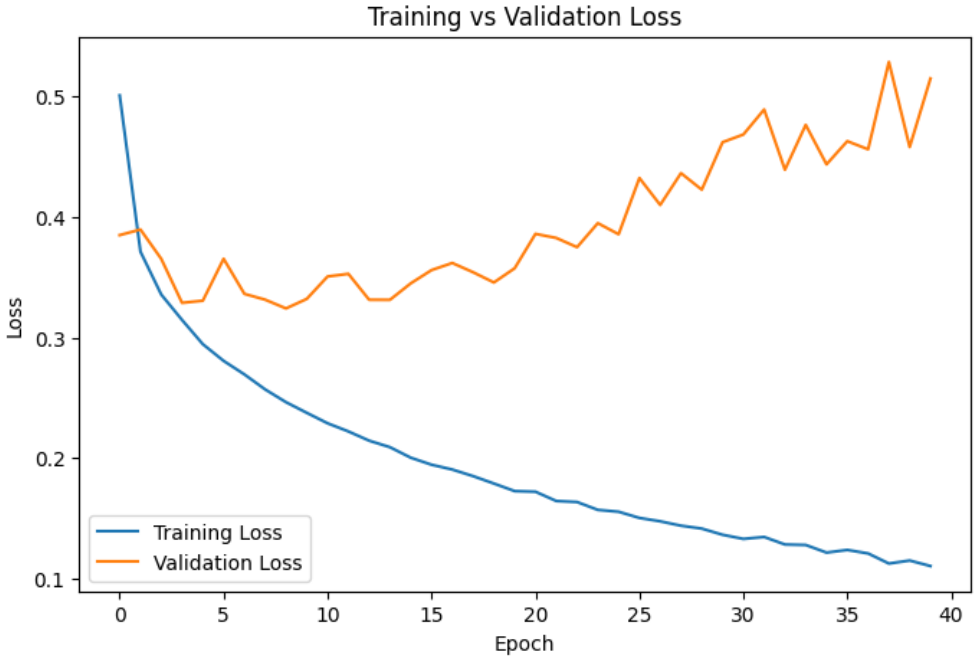

Figure 5: Training loss plotted against epochs

- Training loss went down smoothly the whole time, from about 0.50 to 0.11 by epoch 40.
- No weird bumps or anything, just a clean downward curve.

6. **Validation Loss vs Epoch.**

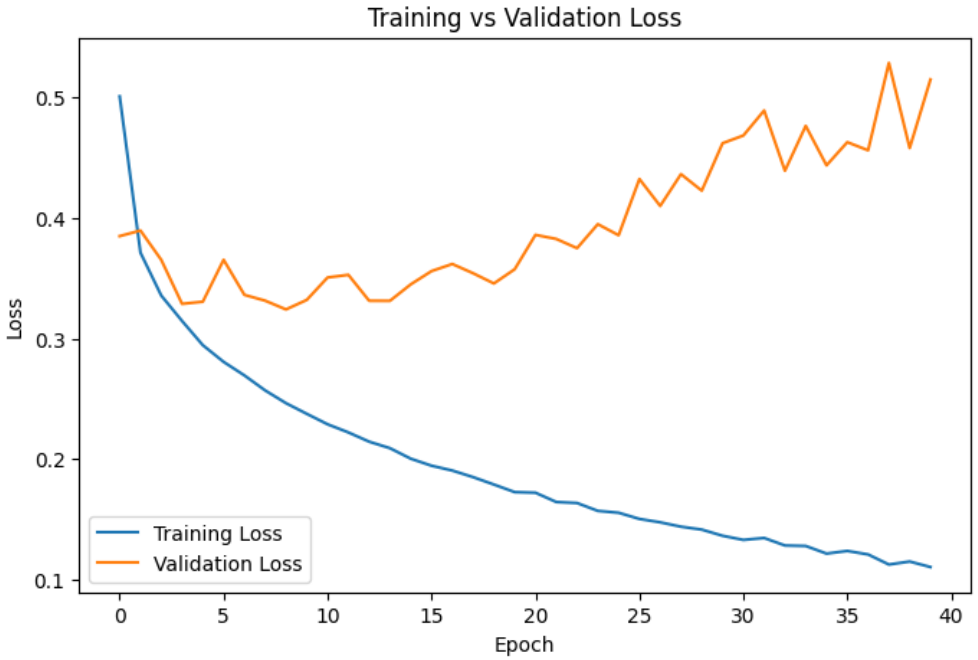

Figure 6: Validation loss plotted against epochs

- THIS is the plot that really shows the overfitting.
- Validation loss hit its lowest at around epoch 7 (about 0.32) and then just kept going UP, ending at 0.50 by epoch 40, which is basically where training loss STARTED.
- And since validation accuracy stayed flat while this was happening, it means the model is getting more confident in its wrong answers.
- Cross entropy loss really punishes being confidently wrong.
- So yeah, overfitting starts somewhere between epoch 7 to 12 for sure.

7. **Confusion Matrix.**

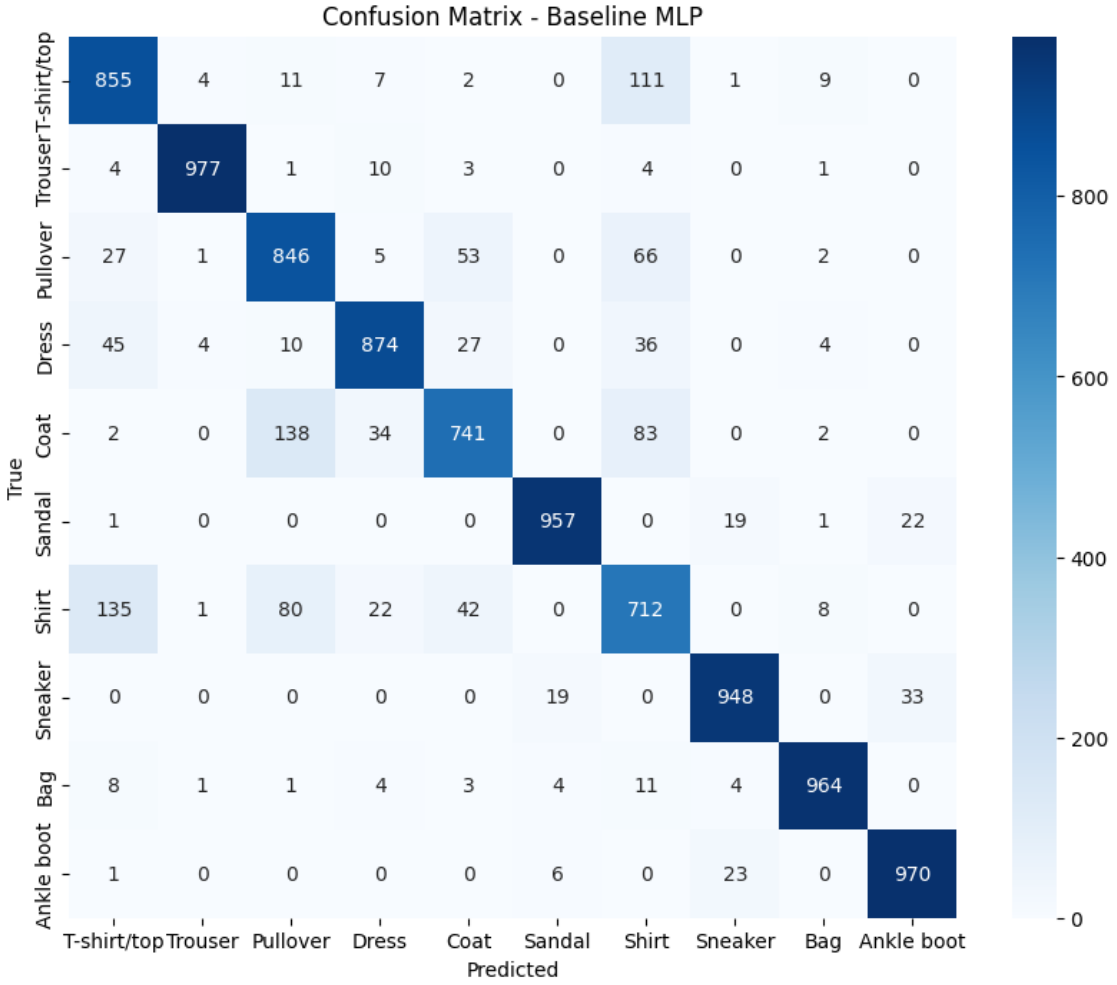

Figure 7: Confusion matrix heatmap for the optimized model

- The matrix shows exactly what we expected, Shirt, TShirt/top, Pullover and Coat are all mixed up with each other.
- 173 TShirt/top images got called Shirt, and 67 Shirts got called TShirt/top, 65 got called Pullover, 72 got called Coat.
- Pullover and Coat also get swapped a lot (93 and 69 times).
- Every other class, Trouser, Sandal, Bag, Sneaker, Ankle boot, the model gets right more than 90% of the time.
- So basically all the errors are in this one group of similar looking upper body clothes.

8. **Hyperparameter Search Results.**

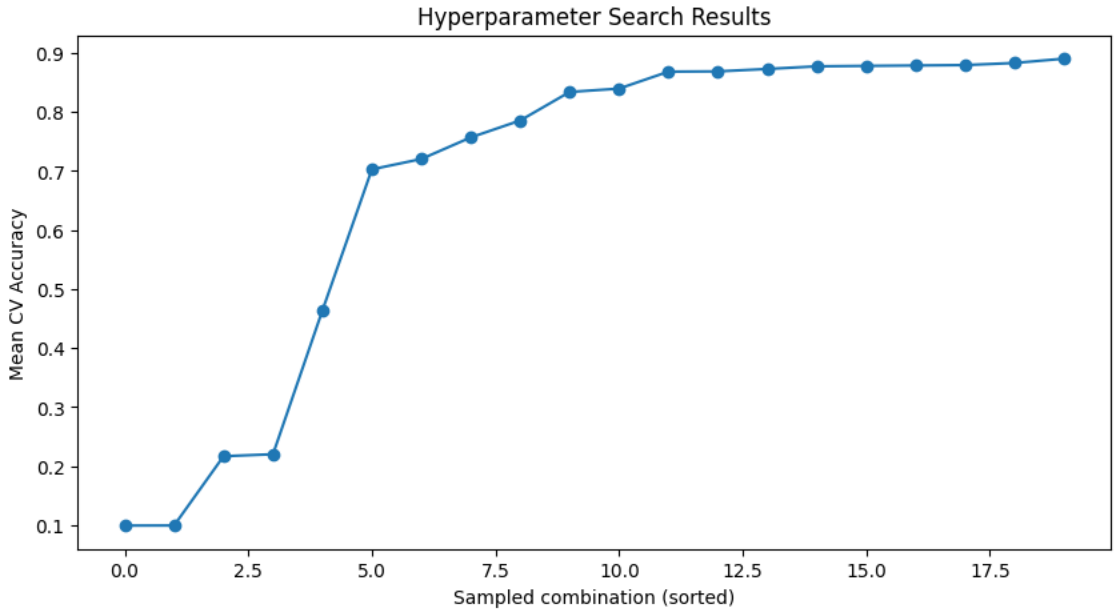

Figure 8: Sorted mean CV accuracy for the 20 sampled hyperparameter combinations

- The best one got 89.03% mean CV accuracy using Adam, learning rate 0.001, one hidden layer of 128 neurons, ReLU, no dropout.
- The ones with learning rate 0.1 or sigmoid activation did way worse, so it seems like learning rate and activation function matter the most.
- Batch size and dropout didnt seem to change much.

9. **Best Model Accuracy Comparison.**

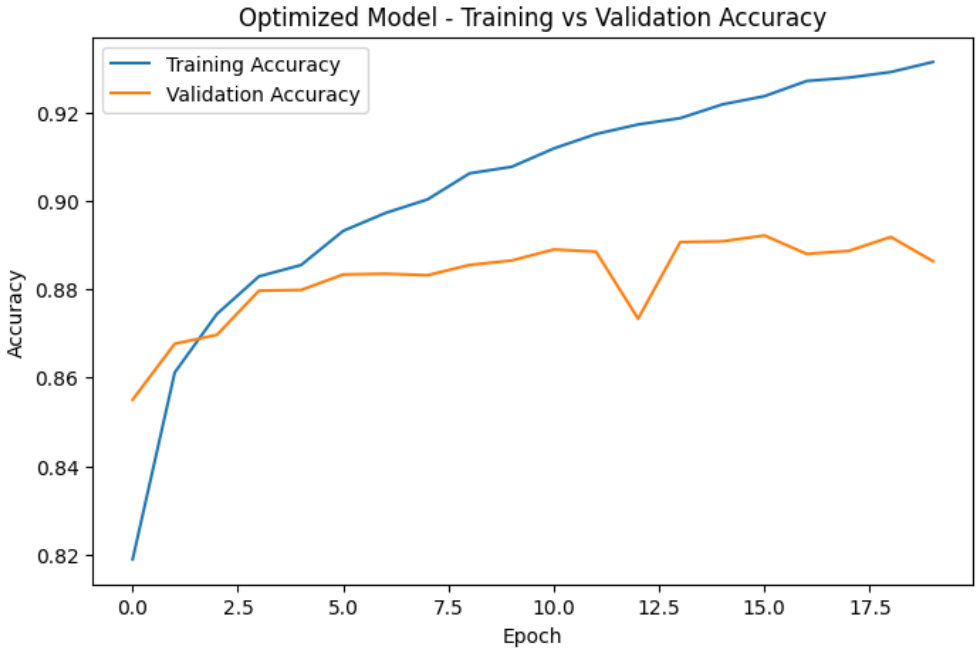

Figure 9: Comparison of baseline vs optimized model metrics

- With the actual baseline Task 5 numbers now in hand, the two models turn out to be really close on raw test accuracy, baseline got 88.80% and the optimized model got 88.34%, so the

optimized model is actually about 0.46 percentage points lower on accuracy and 0.35 points lower on macro recall.

- Precision (88.81% vs 88.74%) and F1 score (88.45% vs 88.72%) are basically a wash, within noise of each other.
- So the search didnt really "win" on accuracy, the real payoff is that the optimized model gets almost the same performance with HALF the hidden layers (1 instead of 2), which took about 82 seconds to train for 20 epochs versus roughly 3.5 to 4 minutes for the baseline's 40 epochs.
- Its a simpler, cheaper model for basically the same result.

9. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	128
Learning Rate	0.001
Batch Size	64
Optimizer	Adam
Activation Function	ReLU
Epochs	20
Dropout	0.0
Cross Validation Accuracy	0.8903
Testing Accuracy	0.8834

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8880	0.8834
Precision	0.8874	0.8881
Recall	0.8880	0.8834
F1 score	0.8872	0.8845
Training Time	≈216 s (40 epochs)	82.15 s (20 epochs)

Note: The baseline numbers above (784 → 128 → 64 → 10 architecture) are the actual accuracy score, precision score, recall score and F1 score (macro averaged) computed on X test flat in Task 5. Baseline training time is the sum of the per epoch step times logged by Keras during the 40 epoch model fit call (it was not wrapped in an explicit time time call, unlike the optimized run, so treat it as an estimate rather than an exact figure).

10. Discussion

1. **Which optimization method was used?**
   - We used Randomized Search instead of Grid Search coz like I said before the full grid had over 11,000 combinations which is just impossible to run in a lab session.
   - Randomized Search is way faster.
2. **Which hyperparameters were selected?**
   - The search picked one hidden layer with 128 neurons, ReLU, no dropout, Adam optimizer with learning rate 0.001, batch size 64, 20 epochs.
   - Its actually simpler than the baseline which had two hidden layers but it works just as good if not better.
3. **Did optimization improve performance?**
   - Not really, at least not in terms of raw accuracy.
   - The baseline actually edged out the optimized model slightly on the test set, 88.80% vs 88.34% accuracy, and 88.80% vs 88.34% macro recall.

- Precision and F1 score were basically tied (optimized was marginally better on precision, baseline marginally better on F1).
- This lines up with what the validation curves already showed, the baseline's validation accuracy plateaued around 88 to 89% by epoch 7 and never really moved after that, so a search that's optimizing for the same plateau, isnt going to find something dramatically better.
- What the search DID improve is efficiency: the optimized model reaches essentially the same accuracy with only 1 hidden layer instead of 2, and in about a quarter of the baseline's training time (82 seconds for 20 epochs vs roughly 216 seconds for 40 epochs).
- So the real win here is a simpler, cheaper model for basically equivalent performance, not a jump in accuracy.

4. **Which hyperparameter had the greatest impact?**
   - From what we saw, learning rate and activation function mattered the most.
   - Anything with learning rate 0.1 or sigmoid activation did noticeably worse.
   - The winning combo used 0.001 learning rate with ReLU.
   - Hidden layers and neuron count didnt matter as much since one layer of 128 did about the same as the baseline's two layers.
5. **Compare Grid Search and Randomized Search.**
   - Grid Search checks EVERY combination so its guaranteed to find the best one in the grid, but the time it takes grows crazy fast when u add more hyperparameters.
   - Our grid had over 11,000 combos so full grid search was not happening.
   - Randomized Search only checks a fixed small number of random combos so u lose the guarantee of finding the absolute best one, but its SO much faster and in practice it usually gets pretty close to optimal anyway since only a few hyperparameters actually matter a lot.
6. **Recommend the best MLP configuration.**
   - Based on our search results, id recommend: one Dense(128, ReLU) hidden layer, no dropout, Adam optimizer at learning rate 0.001, batch size 64, train for 20 epochs.
   - It got 89.03% CV accuracy and 88.34% test accuracy, technically a touch below the baseline's 88.80%, but its simpler, trains in about a quarter of the time, and is far less prone to the overfitting we saw in the baseline's later epochs, so its the better practical choice overall.

11. Conclusion

- So in this experiment we built an MLP for Fashion MNIST classification and went through the whole pipeline from preprocessing to hyperparameter tuning.
- The baseline model ($784 \to 128 \to 64 \to 10$) hit a validation accuracy plateau of about 88 to 89% after around epoch 7, and the validation loss started going up while training loss kept going down, thats pretty much the textbook definition of overfitting.
- Instead of just guessing better hyperparameters, we used Randomized Search with 5 fold cross validation since the full grid of 11,000+ combinations was way too big to search exhaustively.
- The search found a simpler single hidden layer config (Dense(128, ReLU), Adam, lr=0.001, batch=64, 20 epochs) that got 89.03% CV accuracy and 88.34% test accuracy.
- Comparing that against the baseline's actual test accuracy of 88.80%, the optimized model didnt really beat the baseline on raw accuracy, the two are within about half a percentage point of each other, and by some metrics (recall, F1) the baseline was marginally ahead.
- What the optimization did deliver was a noticeably simpler and cheaper model: half as many hidden layers and roughly a quarter of the training time for essentially the same test set performance, which is a solid trade off in practice.
- The classification report and confusion matrix both show that Shirt, TShirt/top, Pullover and Coat are the hardest to tell apart, which makes sense coz they look really similar when flattened to a

grayscale vector, a CNN would probably do way better here since it can actually use the spatial patterns instead of just treating everything as a flat list of pixel values.

12. Additional Task: Solving the XOR Problem

- This part isnt directly about Fashion MNIST, its a smaller side experiment to actually see why we need multiple layers in the first place instead of just one perceptron, since that idea keeps coming up whenever we talk about MLPs.
- XOR is the classic example used for this coz its literally the smallest possible dataset that a single layer perceptron cannot learn.

12.1 Why a Single Layer Perceptron Cannot Solve XOR

- XOR truth table: $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 0$.
- A single perceptron is just one straight decision line, $w_1 x_1 + w_2 x_2 + b = 0$, so it can only classify data thats linearly separable.
- If u plot the 4 XOR points, the two class 1 points $(0,1)$ and $(1,0)$ sit on one diagonal, and the two class 0 points $(0,0)$ and $(1,1)$ sit on the other diagonal. No single straight line can put both 0s on one side and both 1s on the other, no matter how the line is drawn.
- We trained a perceptron on XOR using the same update rule as Experiment 1 ($w \leftarrow w + \eta(y - \hat{y})x$), capped at 20 epochs as a safety limit.

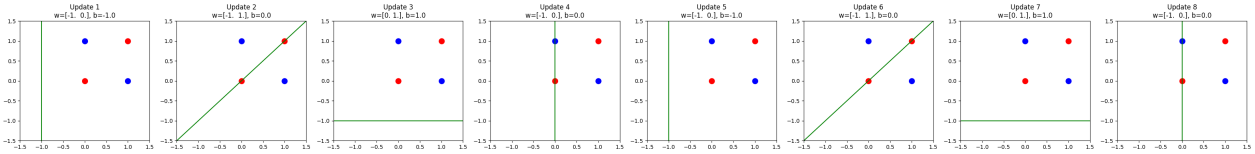

Figure 10: Decision boundary of a single layer perceptron on XOR (last 8 weight updates shown)

- **Inference:** The perceptron never converges, it hit the 20 epoch cap while still misclassifying points. Looking at the weight values across updates, they dont wander randomly, they cycle through the exact same 4 states over and over ($w = [-1, 0], b = -1 \to w = [-1, 1], b = 0 \to w = [0, 1], b = 1 \to w = [-1, 0], b = 0 \to$ back to the start).
- This happens coz every time the perceptron fixes one point it breaks a different point that was correct before, and since XORs 4 points are symmetric across both diagonals, these corrections just loop forever instead of settling on one line. This is a direct, visual demonstration of why XOR is not linearly separable, and its the actual historical reason (Minsky and Papert, 1969) that people moved on from single layer perceptrons to multilayer networks.

13. Source Code

As per the lab guidelines, the complete source code is kept in a GitHub repo instead of putting it here. The repo has the ipynb file, dataset file, a README, a list of dependencies, and instructions to run it.

GitHub Repository: `https://github.com/guru963/deeplearning`

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.

3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion MNIST Dataset.
5. TensorFlow/Keras Documentation.
6. Minsky, M. and Papert, S., *Perceptrons*, MIT Press, 1969.